PROJECT REPORT

AI Resume Builder with ATS Optimization

An AI-Powered Full-Stack Web Application for Building, Scoring and Tailoring ATS-Friendly Resumes

Submitted by: [Your Name] | [Roll Number]

Course: [Course / Branch]

Department: [Department Name]

College: [College / Institute Name]

Guide: [Faculty / Project Guide Name]

Academic Year: [2025 – 2026]

Technology Stack

Frontend: Next.js 16 + React 19 + TypeScript + Tailwind CSS

Backend: Python FastAPI + SQLAlchemy (async) + Pydantic v2

Database: Supabase PostgreSQL / local SQLite fallback

AI Provider: OpenRouter (free LLMs) via OpenAI-compatible API

Export: WeasyPrint / Playwright (PDF) + python-docx (DOCX)

Abstract

Many job seekers, especially freshers and students, are rejected before a human recruiter ever reads their resume. The reason is the **Applicant Tracking System (ATS)** — software that automatically parses, filters and ranks resumes before they reach a recruiter. Resumes that use complex tables, multi-column layouts, non-standard headings, weak action verbs or missing keywords are silently discarded, no matter how qualified the candidate is.

This project, **AI Resume Builder with ATS Optimization**, is a full-stack web application that helps a user create a resume which is readable by ATS software and highly targeted to a specific job. It provides a guided resume builder with a live preview, a rule-based ATS score analyzer (0–100) with itemized fixes, a job-description (JD) matcher that extracts required skills using an AI model and semantically matches them against the resume, and an AI career toolkit (summary generation, bullet rewriting, cover letters, interview preparation, recruiter audit and cold-outreach messages). Finally, the resume can be exported as a clean ATS-friendly **PDF**, **DOCX**, plain text or HTML.

The application follows a three-tier architecture: a Next.js/React frontend (presentation), a FastAPI backend (business logic and REST API), and a PostgreSQL/SQLite database (persistence). AI features are integrated through OpenRouter's free language models with structured JSON outputs, a fallback-model chain, and anti-hallucination guardrails so the AI cannot invent qualifications, companies or metrics that the user never provided.

Table of Contents

1. Introduction

- 1.1 Overview
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Project

2. Literature Survey and Background

- 2.1 What is an ATS?
- 2.2 Why Resumes Get Rejected
- 2.3 Existing Systems and Their Limitations
- 2.4 Proposed Solution

3. System Analysis

- 3.1 Existing System
- 3.2 Proposed System
- 3.3 Feasibility Study
- 3.4 Functional Requirements
- 3.5 Non-Functional Requirements
- 3.6 Hardware and Software Requirements

4. System Design

- 4.1 System Architecture

4.2 Module Design
4.3 Data Flow
4.4 Database Design
4.5 API Design
4.6 Frontend Page Map
4.7 AI Pipeline Design
4.8 ATS Scoring Algorithm
4.9 Security and AI Guardrails

5. Implementation

5.1 Project Structure
5.2 Backend Implementation
5.3 Frontend Implementation
5.4 Database Implementation
5.5 AI Service Implementation
5.6 Resume Parsing Implementation
5.7 Export Implementation

6. Testing

6.1 Testing Strategy
6.2 Test Cases
6.3 Automated Tests

7. Results and Discussion

7.1 Features Achieved
7.2 Advantages
7.3 Limitations

8. Conclusion and Future Scope

References

Appendix A: API Endpoints

Appendix B: Environment Variables

Appendix C: How to Run the Project

1. Introduction

1.1 Overview

The **AI Resume Builder with ATS Optimization** is a web application that combines a structured resume editor, a deterministic ATS scoring engine, an AI writing assistant and a job-application tracker into one platform. The user fills in their details (contact, summary, experience, education, skills, projects, certifications) through a tabbed builder, sees a live preview, receives an ATS compatibility score with concrete fixes, tailors the resume to a pasted job description, and downloads the final resume in PDF or DOCX format.

The system is designed around one central idea: *a resume must first pass the machine, then impress the human*. Every generated and exported artifact therefore uses a single-column layout, standard section headings, web-safe fonts, selectable text and standard date formats.

1.2 Problem Statement

Most candidates write resumes manually in word processors that encourage tables, graphics, icons, text boxes and multi-column layouts. These look attractive to humans but are poorly parsed by ATS software. In addition, candidates rarely know which keywords a specific job description requires, and they often use weak phrases such as "responsible for" or "worked on" instead of measurable, action-oriented achievements. There is no affordable, easy-to-use tool that simultaneously:

- builds a structured, ATS-safe resume,
- scores it objectively and explains every deduction,
- matches it against a real job description, and
- helps generate targeted content without fabricating facts.

1.3 Objectives

1. To provide a guided resume builder with a live preview and three ATS-safe templates.
2. To implement a rule-based ATS scoring engine (0–100) with category-wise issues and fixes.
3. To parse uploaded PDF/DOCX resumes and analyze their layout (tables, columns, headings).
4. To integrate an LLM (through OpenRouter) for summary generation, bullet improvement, skill suggestions, JD analysis, cover letters, interview preparation, recruiter audit and outreach messages.
5. To semantically match job-description keywords with resume skills using a synonym/alias engine.
6. To persist resumes, ATS scans, job descriptions and job applications in a database.
7. To export resumes as ATS-clean PDF, DOCX, plain text and HTML.
8. To protect users from AI hallucination through structured outputs and guardrails.

1.4 Scope of the Project

The current scope covers a single user working anonymously in a browser. There is no authentication; a stable browser-generated `userId` scopes every resume and application. The system supports PDF and DOCX resume parsing, three export formats (PDF, DOCX, text/HTML), and ten front-end pages covering the complete job-search workflow. The AI features depend on an external OpenRouter API key; when the AI is unavailable, the system degrades gracefully for bullet improvement and returns clear error messages

for other features. The database defaults to a local SQLite file but can be pointed at Supabase PostgreSQL for production.

2. Literature Survey and Background

2.1 What is an ATS?

An Applicant Tracking System (ATS) is software used by employers to receive, store, search and rank job applications. When a candidate uploads a resume, the ATS extracts the text, segments it into sections (contact, summary, experience, education, skills), and indexes the keywords. Recruiters then search and filter candidates by keywords, job titles and years of experience. If the ATS fails to extract the text correctly, the candidate becomes invisible regardless of their actual qualifications.

2.2 Why Resumes Get Rejected

Problem	Effect on ATS
Tables and text boxes	Content is skipped or read out of order
Multi-column layouts	Columns get merged into unreadable text
Images/icons with text	Text cannot be extracted at all
Non-standard headings (e.g. "My Journey")	Section detection fails
Missing keywords from the JD	Resume is ranked below keyword-rich resumes
Weak phrases ("responsible for")	Lower relevance/quality score
Non-standard dates (e.g. "Summer 22")	Experience timelines are misread
No quantified results	Lower ranking compared with metric-driven bullets

2.3 Existing Systems and Their Limitations

Commercial resume builders (e.g. generic online editors) provide templates but usually do not explain ATS scoring, do not match against a specific job description, and lock advanced features behind subscriptions. Manual word-processor templates require design knowledge and often break ATS parsing. Standalone ATS-checker websites return a score but do not help the user fix content or tailor it to a job. Most importantly, many AI resume tools freely invent experience and metrics, which is unethical and easily caught in interviews. This project addresses all of these gaps in one free, self-hostable application.

2.4 Proposed Solution

A single full-stack application that (a) enforces ATS-safe structure by design, (b) scores the resume deterministically so the result is explainable, (c) uses an LLM only for language generation with strict structured-output validation and anti-hallucination checks, and (d) manages the complete application pipeline from resume creation to job tracking.

3. System Analysis

3.1 Existing System

In the existing manual process, a candidate creates a resume in a word processor, exports it to PDF, and applies to jobs. The candidate has no reliable way to know whether the ATS parsed the file correctly, which keywords are missing, or why an application was rejected. Feedback is rare, and improvements are guesswork.

3.2 Proposed System

The proposed system replaces guesswork with measurable feedback. The user builds a structured resume (stored as JSON), receives an explainable ATS score, sees matched and missing JD keywords, uses AI to improve wording without inventing facts, and exports a clean, parser- friendly document. A tracker records every application stage.

> **Key design principle:** the resume is stored as structured data first and rendered to a document second. This guarantees that all exports are consistent, and it allows both the scoring engine and the AI to operate on well-defined fields instead of free text.

3.3 Feasibility Study

Technical Feasibility

All chosen technologies are open-source and mature. Next.js and React provide a modern, component-based UI. FastAPI offers high-performance async REST APIs with automatic OpenAPI documentation. SQLAlchemy's async engine supports both SQLite and PostgreSQL without code changes. OpenRouter exposes free LLMs through an OpenAI-compatible interface. PDF and DOCX generation use well-established Python libraries. The project therefore requires no proprietary or paid component for local development.

Economic Feasibility

The stack is free. AI usage can be limited to free models (suffix `:free`) on OpenRouter. SQLite requires no server. For production, Supabase offers a free Postgres tier. The only optional cost is a hosting plan for the backend, which can run on a small container.

Operational Feasibility

The UI is a standard web application with a guided tabbed builder, so no training is required. The API is stateless and horizontally scalable. The anonymous user-id design removes the friction of account creation for a demo or academic deployment.

3.4 Functional Requirements

ID	Requirement
FR-01	The system shall allow a user to create, edit, save, load, duplicate and delete resumes.

FR-02	The builder shall capture contact, summary, experience, education, skills, projects and certifications.
FR-03	The system shall show a live resume preview and three templates (modern, classic, minimal).
FR-04	The system shall compute an ATS score (0–100) with issues, severity, category and suggested fixes.
FR-05	The system shall parse an uploaded PDF/DOCX, extract text and sections, and detect tables and multi-column layout.
FR-06	The system shall analyze a job description and list matched and missing skills with a match percentage.
FR-07	The system shall generate a professional summary, improve bullets, and suggest skills using AI.
FR-08	The system shall tailor the resume to a job description without inventing facts.
FR-09	The system shall generate cover letters, interview questions with STAR answers, recruiter audits and outreach messages.
FR-10	The system shall parse raw resume/LinkedIn text into structured fields.
FR-11	The system shall export the resume as PDF, DOCX, plain text and HTML.
FR-12	The system shall track job applications across five stages: wishlist, applied, interview, offer, rejected.
FR-13	The system shall persist ATS scans and job descriptions for later analysis.

3.5 Non-Functional Requirements

- **Performance:** deterministic ATS scoring is instant; AI calls use a 60-second timeout with model fallback.
- **Reliability:** if the database is unavailable, the health endpoint reports "degraded"; ATS persistence failures never block returning a score.
- **Usability:** responsive Tailwind UI, live preview, loading states, error banners and confirmation modals.
- **Security:** every resume and application is scoped by the caller's `userId`; secrets are read from environment variables.
- **Portability:** runs on Windows/Linux with SQLite, or with Supabase PostgreSQL in production.
- **Maintainability:** layered backend (API → services → models), typed frontend, Pydantic schemas, and automated tests.

3.6 Hardware and Software Requirements

Type	Requirement
Processor	Any modern dual-core CPU (quad-core recommended for PDF rendering)

RAM	4 GB minimum, 8 GB recommended
Disk	~500 MB for Node modules, Python packages and project files
Operating System	Windows 10/11, Linux or macOS
Runtime	Python 3.11+ and Node.js 20+
Browser	Chrome, Edge, Firefox or Safari (latest)
Optional	Docker Desktop (for local PostgreSQL + pgAdmin)
Network	Internet access for OpenRouter AI calls

4. System Design

4.1 System Architecture

The application uses a classic **three-tier architecture** with an additional external AI service tier.

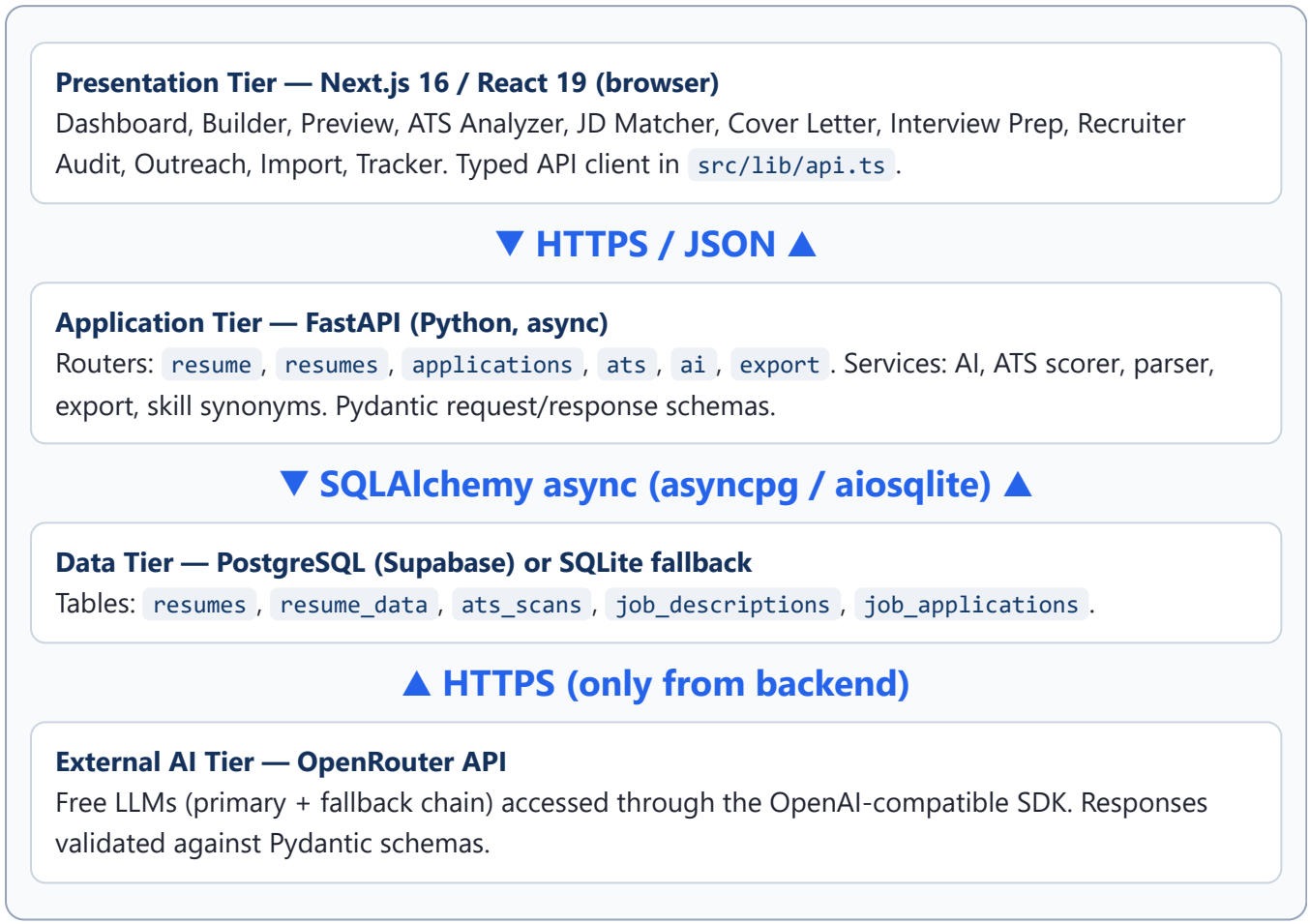


Figure 4.1 — System architecture of the AI Resume Builder

4.2 Module Design

Module	Location	Responsibility
Resume Parser	`backend/app/services/resume_parser.py`	Extract text/sections from PDF/DOCX; detect tables and columns.
ATS Scorer	`backend/app/services/ats_scorer.py`	Rule-based scoring on structured data and on parsed text; date/header standardization.
AI Service	`backend/app/services/ai_service.py`	All LLM prompts, structured outputs, fallback models, guardrails, local degradation.
Skill Synonyms	`backend/app/services/skill_synonyms.py`	Canonical skill aliases, hard/soft categorization, word-boundary matching.
Export Service	`backend/app/services/export_service.py`	ATS-clean HTML template, PDF via WeasyPrint/Playwright, DOCX via python-docx, plain text.

Database Layer	`backend/app/database.py`, `models/resume.py`	Async engine, session factory, table creation, repository helpers.
Schemas	`backend/app/schemas/resume.py`	Pydantic models for resume data, requests, AI structured outputs.
API Routers	`backend/app/api/*.py`	HTTP endpoints, validation, error mapping, persistence orchestration.
API Client	`frontend/src/lib/api.ts`	Typed fetch wrappers for resumes, AI, ATS, export and applications.
Builder	`frontend/src/app/builder/page.tsx`	Tabbed editor, local draft autosave, AI diff modal, save/update.
Preview/Export	`frontend/src/app/preview/page.tsx`	Final rendering, ATS diagnostic, PDF/DOCX download.

4.3 Data Flow

Flow 1 — Build and save a resume

1. User fills the builder tabs; state is kept in React and mirrored to `localStorage` as a draft (400 ms debounce).
2. On save, the frontend sends `POST /api/resumes` or `PUT /api/resumes/{id}` with the `userId`, title, template and structured data.
3. FastAPI validates the payload with Pydantic, then the repository writes the `resumes` row and the `resume_data` JSONB row.
4. The API returns the serialized resume; the frontend redirects to `/builder?id=...`.

Flow 2 — ATS scan of an uploaded file

1. User uploads a PDF/DOCX on the Analyzer page.
2. Frontend calls `POST /api/resume/parse` as `multipart/form-data`.
3. Backend extracts text (pdfplumber/python-docx), detects sections, tables and column layout.
4. The text-based ATS scorer returns a score, issues and passed checks.
5. The UI displays the score card and can hand the extracted text to the JD Matcher.

Flow 3 — JD matching and tailoring

1. User pastes a job description; the frontend calls `POST /api/ai/analyze-jd`.
2. The AI extracts required/preferred skills and keywords; the synonym engine matches them against the resume text.
3. The API returns matched/missing skills and a match percentage; the user may insert missing keywords.
4. Optionally, `POST /api/ai/tailor-resume` rewrites wording, and guardrails restore any fabricated entities.

4.4 Database Design

The authoritative schema is defined as SQLAlchemy models in `backend/app/models/resume.py` and is created automatically on startup. On PostgreSQL the JSON columns become `JSONB`; on SQLite they

remain JSON text.

Table	Key Columns	Purpose
`resumes`	id (PK), user_id (index), title, target_role, template, parent_id (index), score, created_at, updated_at	Resume metadata. `parent_id` links a tailored clone to its source variant.
`resume_data`	id (PK), resume_id (FK → resumes.id, unique, ON DELETE CASCADE), data (JSONB), timestamps	Structured resume body: contact, summary, experience, education, skills, projects, certifications.
`ats_scans`	id (PK), resume_id (FK, index), score, matched_keywords, missing_keywords, suggestions, created_at	History of ATS scoring results for a resume.
`job_descriptions`	id (PK), user_id (index), target_role, raw_text, parsed_skills (JSONB), created_at	Saved job descriptions and their extracted skills.
`job_applications`	id (PK), user_id (index), company_name, role_title, location, salary, status (index), date_applied, resume_id (FK, SET NULL), notes, timestamps	Kanban application pipeline; optionally linked to a resume variant.

Figure 4.2 — Database tables and relationships

Relationships: one `Resume` has exactly one `ResumeData` (1:1) and many `AtsScan` records (1:N). A `JobApplication` may reference one `Resume` (N:1). `Resumes` also self-reference through `parent_id` for variants.

4.5 API Design

The backend exposes a JSON REST API under `/api`. It is documented automatically by FastAPI at `/docs` (Swagger UI) and `/redoc`. The complete endpoint list is given in Appendix A.

Group	Base Path	Operations
Health	`/api/health`	Database connectivity and provider check
Parsing	`/api/resume`	Upload PDF/DOCX → text, sections, layout, ATS analysis
Resume CRUD	`/api/resumes`	Create, list, get, update, delete, clone, insert keywords
ATS	`/api/ats`	Score structured resume data
AI	`/api/ai`	Bullets, summary, skills, JD, tailor, cover letter, interview, raw parse, audit, outreach
Export	`/api/export`	PDF, DOCX, text, HTML
Applications	`/api/applications`	List, create, update, delete tracked jobs

4.6 Frontend Page Map

Route	Page	Function

`/`	Dashboard	Stats, resume list, search, duplicate/delete, toolkit shortcuts
`/builder`	Builder	Tabbed editor, live draft autosave, AI actions with diff modal
`/preview`	Preview	Rendered resume, ATS diagnostics, template switch, PDF/DOCX export
`/analyzer`	ATS Scan	Upload PDF/DOCX, view score card and extracted text
`/matcher`	JD Matcher	Paste JD, view matched/missing skills, insert keywords
`/import`	Import	Paste raw resume/LinkedIn text → AI auto-fills structured fields
`/cover-letter`	Cover Letter	Generate and copy a tailored cover letter
`/interview-prep`	Interview Prep	Generate behavioral/technical questions with STAR answers
`/recruiter-audit`	Recruiter Scan	6-second cold-read audit with red flags and top fixes
`/outreach`	Outreach	Connection note, cold email, referral and follow-up messages
`/tracker`	Job Pipeline	Kanban board across five application stages

4.7 AI Pipeline Design

All AI features share a single resilient pipeline in `ai_service.py`:

1. **Prompt construction:** a role-specific system prompt plus a user prompt containing only the user's own data.
2. **Structured output:** the model is asked to return JSON; the response is parsed (code-fence stripping and regex extraction) and validated with a Pydantic model.
3. **Model fallback chain:** primary model first (default `deepseek/deepseek-chat-v3-0324:free`), then configured fallbacks (Llama 3.3 70B free, Gemini 2.0 Flash Lite free). On 429/timeout/5xx/empty response, the next model is tried with a short backoff.
4. **Graceful degradation:** if all models fail for bullet improvement, a local rule-based rewrite strips passive phrases and inserts a strong verb, so the UI still works offline.
5. **Guardrails:** see section 4.9.

4.8 ATS Scoring Algorithm

Structured scoring (builder data)

The resume starts at 100 points and deductions are applied for each missing or weak element. The same deductions are reported as issues with a severity (`error` / `warning`), a category and a concrete fix.

Check	Deduction	Severity
Full name missing	−10	error
Email missing	−10	error
Phone missing	−5	error
Professional summary missing	−10	warning

Experience section missing/empty	−15	error
Experience entries without bullets	−5	warning
Experience entries missing dates	−5	error
Irregular date format	−5 (per category)	warning
Education missing	−10	warning
Skills missing	−10	warning
No quantified achievement in bullets	−5	warning
Passive phrasing in bullets	−5	warning

Passed checks (e.g. "Contact information present", "Quantified achievements present") are also returned so the user sees both strengths and weaknesses. The final score is clamped to 0–100 and summarized as "well-structured" (≥80), "some issues" (60–79) or "needs significant improvements" (<60).

Text scoring (uploaded file)

For an uploaded file the engine checks the presence of the six standard sections (Summary, Experience, Education, Skills, Projects, Certifications), total content length, non-standard heading-like lines, table detection and multi-column detection. Each failure deducts points and produces a fix. The PDF parser identifies tables with `pdfplumber.find_tables()` and estimates columns by analysing the distribution of word x-coordinates.

4.9 Security and AI Guardrails

- **Ownership scoping:** every resume/application query filters by `user_id`, preventing one browser identity from reading another's data.
- **Secrets in environment:** API keys and database URLs are read from `.env` and never shipped to the browser (only `NEXT_PUBLIC_API_URL` is public).
- **Anti-hallucination (tailor):** the original resume JSON is the source of truth. After the AI responds, the backend restores contact, education, projects and certifications unchanged; rejects fabricated companies; drops skills that were not in the original resume (or an exact synonym); and records `guardrailWarnings`.
- **Fact preservation (bullets/summary):** prompts explicitly forbid inventing metrics, tools, titles or credentials; metrics are omitted rather than fabricated.
- **Validation:** every AI response must pass Pydantic validation or the model is retried/failed safely.
- **Error mapping:** authentication, rate-limit, quota and upstream errors are mapped to meaningful HTTP status codes and user-friendly messages.

5. Implementation

5.1 Project Structure

```
AI RESUME BUILDER WITH ATS/
├── frontend/                    # Next.js web app
│   ├── src/app/                 # Pages (dashboard, builder, analyzer, ...)
│   ├── src/components/          # UI components + ResumeDocument
│   ├── src/hooks/useResumes.ts  # Resume loading hooks
│   ├── src/lib/api.ts           # Typed API client
│   ├── src/lib/resume-text.ts   # Plain-text rendering + page estimate
│   ├── src/lib/user-id.ts       # Anonymous browser identity
│   └── src/types/resume.ts      # Shared TypeScript types
├── backend/                     # Python FastAPI server
│   ├── main.py                  # App factory, CORS, routers, health
│   ├── app/database.py          # Async engine + session + init_db
│   ├── app/models/resume.py     # SQLAlchemy models + repositories
│   ├── app/schemas/resume.py    # Pydantic request/response schemas
│   ├── app/api/                 # Routers (resume, resumes, ats, ai, export, applications)
│   ├── app/services/            # AI, ATS scorer, parser, export, skill synonyms
│   ├── tests/                   # pytest test-suite
│   ├── Dockerfile               # Production image (WeasyPrint libs included)
│   └── requirements.txt         # Python dependencies
├── database/                    # Local Postgres compose + seed + docs
├── README.md
└── AGENTS.md
```

5.2 Backend Implementation

Application entry point (`main.py`): creates the FastAPI app, adds CORS for localhost and configured origins, registers six routers, and defines `/api/health`. A lifespan handler calls `init_db()` on startup and `close_db()` on shutdown. The health endpoint probes the database with `SELECT 1` and reports the provider (`sqlite` or `postgres`).

Database layer (`app/database.py`): reads `DATABASE_URL`, defaults to `sqlite+aiosqlite:///./resume_builder.db`, and creates an async engine. SQLite gets `check_same_thread=False`; PostgreSQL gets a pool of 5 with 10 overflow. Sessions are provided to routes through a FastAPI dependency.

Repository layer (`app/models/resume.py`): contains all CRUD operations as async functions: `create_resume`, `get_resume_for_user`, `list_resumes`, `update_resume`, `delete_resume`, `create_ats_scan`, `save_job_description` and the job-application helpers. Every function that accesses user data receives `user_id` for ownership enforcement.

Validation (`app/schemas/resume.py`): defines the resume object graph (`ContactInfo`, `WorkExperience`, `Education`, `Skill`, `Project`, `Certification`, `ResumeData`), all request models, and structured AI output models.

5.3 Frontend Implementation

The frontend uses the Next.js App Router with client components for interactive pages. Each page is a self-contained module under `src/app`. Shared UI primitives (Button, Card, Field, Modal, Chip, Loading, ErrorBanner, AtsGauge) live in `src/components/ui`, so styling and behaviour stay consistent.

- **State:** React `useState` / `useMemo` / `useCallback`; no external state library is needed.
- **Identity:** `getUserId()` creates a UUID in `localStorage` once and reuses it, so saves survive reloads without login.
- **Draft autosave:** the builder mirrors unsaved edits to `localStorage` with a 400 ms debounce; the preview can render a draft before it is saved.
- **AI diff modal:** AI changes are never applied blindly. The original and proposed text are shown side by side with Accept/Regenerate actions.
- **Types:** every API response has a matching TypeScript interface in `src/types/resume.ts`.

5.4 Database Implementation

Tables are created automatically by `Base.metadata.create_all` inside `init_db()`, so no migration tool is required for the MVP. A local PostgreSQL can be started with Docker Compose (`database/docker-compose.yml`), which also launches pgAdmin. `database/seed.py` is idempotent: it creates the schema and inserts a demo resume and job description for the `demo-user` account only if that user has no resumes.

> **Production database:** set `DATABASE_URL=postgresql+asyncpg://postgres:<password>@<host>:6543/postgres?sslmode=require` in `backend/.env`. On first boot the five tables are created automatically.

5.5 AI Service Implementation

The AI service exposes asynchronous functions used by the AI router:

Function	Endpoint	Output
`improve_bullet_point`	/api/ai/improve-bullet	Rewritten bullet, passive-phrase flags, model used
`generate_summary`	/api/ai/generate-summary	3–4 sentence professional summary
`suggest_skills`	/api/ai/suggest-skills	10–15 canonical skills for a role
`extract_jd_keywords`	/api/ai/analyze-jd	Required/preferred skills, keywords, hard/soft split
`tailor_full_resume`	/api/ai/tailor-resume	Rewritten summary/bullets/skills + guardrail warnings
`generate_cover_letter`	/api/ai/generate-cover-letter	Structured letter sections + full text
`generate_interview_prep`	/api/ai/interview-prep	Behavioral/technical questions with STAR answers
`parse_raw_resume_text`	/api/ai/parse-raw-resume	Fully structured `ResumeData`
`generate_recruiter_audit`	/api/ai/recruiter-audit	First impression, callback probability, red flags, top fixes
`generate_outreach_messages`	/api/ai/generate-outreach	Connection note, cold email, referral, follow-up

The **Google XYZ formula** ("Accomplished [X] as measured by [Y], by doing [Z]") is used for bullet rewriting, and a list of passive phrases ("responsible for", "worked on", "helped with", etc.) is detected locally and merged with the model's own flags.

5.6 Resume Parsing Implementation

PDF text and tables are extracted with `pdfplumber`; DOCX text is extracted with `python-docx`. The parser splits the text into sections by matching standard headings at the start of a line, and detects non-standard headings by checking heading-like lines (all-caps or title-case, short, no terminating punctuation) against the canonical set. Column detection compares each word's x-coordinate to the median; if more than 5% of words are far from the median, the layout is considered multi-column.

5.7 Export Implementation

PDF: the resume is rendered into a single-column HTML template (Arial/Helvetica, 10 pt, Letter page, 0.55–0.6 in margins, standard headings, selectable text). WeasyPrint is tried first; if its native GTK libraries are unavailable (common on Windows/macOS), Playwright with headless Chromium is used as a fallback. Both produce a real text layer that ATS parsers can read.

DOCX: generated with `python-docx` using headings, bold company names, italic roles and the `List Bullet` style, keeping the document simple and parseable.

Plain text / HTML: a copy-pasteable text version for legacy ATS text boxes and an HTML version for web preview.

6. Testing

6.1 Testing Strategy

The backend is tested with **pytest** using an in-memory SQLite database configured through `configure_database()`, so tests never touch real data. AI calls are monkeypatched, which makes the test-suite deterministic and offline. Frontend correctness is checked with the Next.js/TypeScript build and ESLint.

6.2 Test Cases

ID	Module	Test Case	Expected Result
TC-01	Resume Parser	Upload a valid PDF/DOCX	Text, sections and layout are returned
TC-02	Resume Parser	Upload an unsupported file	HTTP 400 with a clear message
TC-03	ATS Scorer	Score a complete resume	Score 100 with passed checks
TC-04	ATS Scorer	Remove contact/summary/skills	Score drops with matching issues and fixes
TC-05	ATS Scorer	Use an irregular date	Warning issued, deduction applied
TC-06	Resume CRUD	Create then list a resume	Resume appears for the same userId only
TC-07	Resume CRUD	Update and delete a resume	Changes persist; delete returns the id
TC-08	Resume CRUD	Access another user's resume	HTTP 404 (not leaked)
TC-09	Variants	Clone a resume	New resume with `parent_id` set
TC-10	AI Service	Improve a passive bullet	Strong-verb rewrite + flags
TC-11	AI Service	AI returns invalid JSON	Fallback/retry then graceful error
TC-12	AI Service	Tailor with a fabricated company	Fabrication removed + warning recorded
TC-13	JD Matcher	Analyze a JD against a resume	Matched/missing skills + match %
TC-14	Export	Export PDF and DOCX	Valid files with correct MIME type
TC-15	Tracker	Create/update/delete an application	Pipeline reflects the change
TC-16	Health	Call `/api/health`	Status ok/degraded + provider

6.3 Automated Tests

The `backend/tests` folder contains dedicated test modules:

- `test_resume_parser.py` — file parsing and section extraction
- `test_ats_scorer.py` — scoring rules, dates, sections
- `test_api_resumes.py` — CRUD, ownership, variants
- `test_ai_service.py` — structured outputs, fallbacks, guardrails
- `test_export.py` — PDF/DOCX/text export

- `test_tracking.py` — job-application pipeline

Run them with:

```
cd backend
python -m pytest tests -q
```

7. Results and Discussion

7.1 Features Achieved

11	25+	10	5
FRONTEND PAGES	REST ENDPOINTS	AI FEATURES	DATABASE TABLES

- Guided resume builder with live preview and three ATS-safe templates.
- Deterministic ATS scoring (0–100) with itemized issues and fixes, for both structured data and uploaded files.
- PDF/DOCX parsing with table and multi-column detection.
- AI summary generation, bullet improvement, skill suggestions, JD analysis and full-resume tailoring.
- Semantic skill matching using a large canonical synonym map (React ↔ React.js, K8s ↔ Kubernetes, etc.).
- Cover letter generation, interview prep with STAR answers, recruiter audit and outreach messages.
- Resume variants (clone with `parent_id`) and one-click missing-keyword insertion.
- Job-application Kanban tracker across five stages.
- ATS-clean PDF (WeasyPrint/Playwright), DOCX (python-docx), plain text and HTML export.
- Persistence on Supabase PostgreSQL or local SQLite with automatic schema creation.

7.2 Advantages

- **Explainable scoring:** every deduction is tied to a specific check and a fix, unlike black-box checkers.
- **Ethical AI:** guardrails prevent fabricated experience and metrics.
- **Resilient AI:** model fallback plus a local degradation path.
- **No lock-in:** open-source stack, free AI models, and a SQLite fallback that needs no server.
- **End-to-end workflow:** from resume creation to tailored application and tracking in one place.
- **Structured-first design:** consistent exports and reliable AI inputs.

7.3 Limitations

- No real authentication; identity is a browser-local anonymous id, so data is not portable across devices.
- AI quality depends on the external provider and on free-model availability/rate limits.
- The ATS score is rule-based and heuristic; it approximates real ATS behaviour but cannot guarantee a specific vendor's result.
- PDF export on Windows/macOS depends on Playwright/Chromium when WeasyPrint's native libraries are missing.
- Section extraction from very unusual resume layouts may be imperfect.
- No server-side pagination or caching for large numbers of resumes (fine for individual use).

8. Conclusion and Future Scope

The AI Resume Builder with ATS Optimization successfully demonstrates how a structured, full-stack web application can remove the guesswork from resume writing. By storing the resume as structured data, scoring it deterministically, enriching it with carefully guarded AI, and exporting a parser-friendly document, the system helps candidates pass automated screening and present themselves clearly to recruiters. The three-tier architecture is clean, portable and free to run, while the automated tests and graceful degradation make it reliable in practice.

Future Scope

- **Authentication:** integrate Supabase Auth or OAuth so resumes sync across devices securely.
- **Real ATS integrations:** import job postings directly from job boards and auto-detect company/role.
- **Collaboration:** share a resume for mentor review with comments and version history.
- **Analytics:** dashboards for application conversion rates by resume variant and keyword coverage.
- **Local/private models:** optional Ollama/self-hosted LLM support for privacy-sensitive users.
- **More templates:** additional ATS-safe templates with accessibility options.
- **Internationalization:** multiple languages, date formats and resume conventions.
- **Mobile app:** a companion React Native application sharing the same API.

References

1. FastAPI Documentation — https://fastapi.tiangolo.com/
2. Next.js Documentation — https://nextjs.org/docs
3. React Documentation — https://react.dev/
4. SQLAlchemy 2.0 Documentation — https://docs.sqlalchemy.org/
5. Pydantic v2 Documentation — https://docs.pydantic.dev/
6. Supabase Documentation — https://supabase.com/docs
7. OpenRouter API Documentation — https://openrouter.ai/docs
8. WeasyPrint Documentation — https://doc.courtbouillon.org/weasyprint/
9. python-docx Documentation — https://python-docx.readthedocs.io/
10. pdfplumber Documentation — https://github.com/jsvine/pdfplumber
11. Tailwind CSS Documentation — https://tailwindcss.com/docs
12. pytest Documentation — https://docs.pytest.org/

Appendix A: API Endpoints

Method	Path	Description
GET	`/api/health`	Health and database connectivity check
POST	`/api/resume/parse`	Upload PDF/DOCX → text, sections, layout, ATS analysis
POST	`/api/ats/score`	Score structured resume data
POST	`/api/ai/analyze-jd`	Extract JD keywords and match against resume text
POST	`/api/ai/improve-bullet`	Rewrite a bullet point with AI
POST	`/api/ai/generate-summary`	Generate a professional summary
POST	`/api/ai/suggest-skills`	Suggest skills for a target role
POST	`/api/ai/tailor-resume`	Rewrite the resume for a job description (guarded)
POST	`/api/ai/generate-cover-letter`	Generate a tailored cover letter
POST	`/api/ai/interview-prep`	Generate interview questions with STAR answers
POST	`/api/ai/parse-raw-resume`	Auto-fill structured fields from pasted text
POST	`/api/ai/recruiter-audit`	6-second recruiter cold-read audit
POST	`/api/ai/generate-outreach`	Generate cold-outreach messages
POST	`/api/export/pdf`	Export resume as PDF
POST	`/api/export/docx`	Export resume as DOCX
POST	`/api/export/text`	Export resume as plain text
POST	`/api/export/html`	Export resume as HTML
POST	`/api/resumes`	Save a resume
GET	`/api/resumes?userId=`	List a user's resumes
GET	`/api/resumes/{id}?userId=`	Get one saved resume
PUT	`/api/resumes/{id}`	Update a saved resume
DELETE	`/api/resumes/{id}?userId=`	Delete a saved resume
POST	`/api/resumes/{id}/clone`	Clone a resume as a variant
POST	`/api/resumes/{id}/insert-keywords`	Insert JD keywords into skills/summary
GET	`/api/applications?userId=`	List job applications
POST	`/api/applications`	Create a job application
PUT	`/api/applications/{id}`	Update a job application
DELETE	`/api/applications/{id}?userId=`	Delete a job application

Appendix B: Environment Variables

Backend — `backend/.env`:

Variable	Purpose
`DATABASE_URL`	Database connection. Defaults to SQLite; use the Supabase Postgres URL in production.
`OPENROUTER_API_KEY`	API key for OpenRouter (free models work with $0 balance).
`OPENROUTER_MODEL`	Primary model, e.g. `deepseek/deepseek-chat-v3-0324:free`.
`OPENROUTER_MODEL_FALLBACKS`	Comma-separated fallback models.
`CORS_ALLOWED_ORIGINS`	Extra allowed origins beyond localhost.
`PORT`	Server port (default 8000).
`APP_URL`	Public app URL used in AI provider headers.
`SUPABASE_URL`, `SUPABASE_KEY`, `SUPABASE_SECRET_KEY`	Optional Supabase credentials for future Auth integration.

Frontend — `frontend/.env`:

Variable	Purpose
`NEXT_PUBLIC_API_URL`	Backend base URL, e.g. `http://localhost:8000`.

> **Security note:** never commit real keys to version control. Keep only placeholder values in `.env.example` and add `.env` to `.gitignore`.

Appendix C: How to Run the Project

1. Backend (FastAPI)

```
cd backend
pip install -r requirements.txt
# create backend/.env and set OPENROUTER_API_KEY and (optionally) DATABASE_URL
python -m uvicorn main:app --reload --port 8000
```

API docs: `http://localhost:8000/docs` | Health: `http://localhost:8000/api/health`

2. Frontend (Next.js)

```
cd frontend
npm install
# create frontend/.env with NEXT_PUBLIC_API_URL=http://localhost:8000
npm run dev
```

Website: `http://localhost:3000`

3. Database (optional)

```
# Local PostgreSQL + pgAdmin
cd database
docker compose up -d

# Then set in backend/.env:
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/resume_builder

# Seed demo data (idempotent)
python database/seed.py
```

If Docker is not available, leave `DATABASE_URL` unset. The backend automatically creates and uses `backend/resume_builder.db` (SQLite).

4. Tests

```
cd backend
python -m pytest tests -q
```

5. Production build (frontend)

```
cd frontend
npm run build
npm run start
```

— End of Report —